MEDINOVA

Producto Integrador de Aprendizaje (PIA)

Componentes de Lógica de Negocios · Proyecto final

Tecnologías: Java 21 · Spring Boot · PostgreSQL · Docker Compose · OpenAPI · GitHub Actions

Nombres:	**Matrículas**:
Oscar Denilson García Vélez	1994273
Carlos Hernan Garza Torres	1986937
Edwin Benjamín Ramírez Nava	1956899

Nombre del curso: Componentes de Lógica de Negocios	**Nombre del profesor:** Miguel Ángel Gómez Marroquin
Módulo: Producto Integrador de Aprendizaje	**Actividad:** Unidad 6 PIA / Reporte
Fecha: 21/10/2026	
Bibliografía: Artefactos base del proyecto MediNova incluidos en el paquete en la Actividad 2 y 3	

Contenido

Contenido

Contenido 2
2. Problema, objetivos y alcance 4
3. Análisis del negocio 4
4. Arquitectura DDD y UML 4
5. Patrones y SOLID 4
6. Reglas esenciales y tardanza 4
7. API y persistencia 4
8. Calidad y automatización 5
9. Contenedores, configuración y despliegue 5
10. Trazabilidad 5
11. Riesgos y pendientes 5
12. Plan de demostración y defensa 5
14. Integración del análisis de negocio 6
14.1 Contexto, situación actual y oportunidad 6
14.2 Actores y responsabilidades 6
14.3 Objetivos y alcance 6
15. Requerimientos del negocio (BRD) 7
15.1 Reglas de negocio refinadas 7
15.2 Requerimientos funcionales 7
15.3 Requerimientos no funcionales 7
16. Historias de usuario y criterios de aceptación Gherkin 9
16.1 Priorización funcional para la demostración 10
17. Proceso de negocio y BPMN 11
17.1 Regla de tardanza 11
17.2 Diagramas de proceso y secuencia 12
18. Arquitectura de software y diseño DDD 16
18.1 Modelo de dominio 16
18.2 Límites y consistencia del agregado 16
18.3 Máquina de estados 16
19. Patrones de diseño y principios SOLID 18
19.1 Justificación de selección 18
20. Registros de decisiones de arquitectura (ADR) 19
ADR-001 — Java 21 y Spring Boot 19
ADR-002 — PostgreSQL como persistencia 19
ADR-003 — Appointment como agregado raíz 19
ADR-004 — Specification composable 19
ADR-005 — State Machine en Appointment 19
ADR-006 — API REST y OpenAPI 19
ADR-007 — Docker Compose y CI 20

21. API REST y persistencia 21
21.1 Persistencia 21
22. Calidad, pruebas y CI/CD 22
22.1 Criterio de cobertura 22
22.2 Pipeline 22
23. Contenerización, configuración y despliegue 23
24. Trazabilidad y matriz de cumplimiento 24
24.1 Lectura de cumplimiento 24
25. Riesgos, seguridad y limitaciones 25
26. Guion de demostración en vivo 26
27. Preguntas técnicas para la defensa 27
28. Conclusiones 28
29. Referencias y artefactos fuente 29
30. Anexo: checklist final de evidencia 30

1. Resumen ejecutivo

MediNova es una clínica privada de tres sucursales con cerca de 35 médicos y horarios variables. El PIA integra el trabajo previo de análisis con una implementación demostrativa en Java 21, Spring Boot y PostgreSQL. El núcleo ejecutable modela el ciclo de cita, validación de agenda, llegada, tardanza, decisión médica explícita, cancelación y no-show. Se incorporan Specifications combinables, máquina de estados, eventos de dominio, API REST/OpenAPI, pruebas automatizadas, Docker Compose y workflow de CI.

2. Problema, objetivos y alcance

La operación AS-IS depende de una agenda compartida y acciones manuales. Esto favorece citas duplicadas, baja visibilidad entre sucursales, cancelaciones sin evidencia uniforme y dificultad para medir no-shows e historial. Los objetivos son reducir duplicidades, mejorar disponibilidad, registrar eventos y excepciones, preservar continuidad de consulta y habilitar indicadores. El alcance y los requerimientos refinados están en docs/BRD.md.

3. Análisis del negocio

Se conserva como base el BRD, historias de usuario, BPMN y glosario aportados en el ZIP. El paquete incluye 20 historias HU-01 a HU-20 con escenarios Gherkin; supera el mínimo de 15. Los actores son paciente, recepcionista, médico, administración y aseguradora. Las reglas pendientes de validación (permisos formales, sobrecupo, reincidencia de no-show, ventanas de cancelación) no se convierten en políticas institucionales inventadas.

4. Arquitectura DDD y UML

Appointment representa el agregado raíz del ciclo de vida de una cita; encapsula invariantes y transiciones. Los identificadores de paciente, médico y sucursal se mantienen como referencias. AppointmentService coordina casos de uso; AppointmentRepository encapsula persistencia; API Controller expone contrato REST. Los diagramas de dominio y secuencia se incluyen en docs/architecture/diagrams y se conservan los diagramas originales en originales/.

Componentes principales: Appointment (agregado), AppointmentService (casos de uso), AppointmentRepository (persistencia), AppointmentController (API), AppointmentSpecifications (reglas composables).

5. Patrones y SOLID

Specification concentra validaciones composables (paciente activo, médico activo, sucursal activa, autorización y horario válido) con AND/OR/NOT. State Machine restringe cambios de estado. Domain Events representan confirmación, llegada, decisión y cancelación. Repository separa persistencia del dominio; Service coordina el caso de uso. La separación API/aplicación/dominio/infraestructura apoya SRP y DIP; las interfaces de repositorio desacoplan servicios del mecanismo de almacenamiento.

6. Reglas esenciales y tardanza

La hora de llegada se compara con el inicio programado y se calculan minutos de tardanza. El sistema no resuelve automáticamente si el paciente debe ser atendido. La persona médica asignada registra ATENDER, ESPERAR o REPROGRAMAR. Una tardanza de 15 minutos o menos activa la consulta al médico según el caso original; retrasos mayores normalmente conducen a ofrecer reagenda, sin sustituir la decisión clínica por una regla automática.

7. API y persistencia

La API expone GET/POST /api/appointments y acciones para confirmar, registrar llegada, registrar decisión, cancelar y marcar no-show. Springdoc genera Swagger UI en /swagger-ui.html. PostgreSQL es la base de datos de Compose; H2 se utiliza en pruebas de integración. Los códigos de respuesta previstos incluyen 201 al crear, 400 por validación, 403 por médico no asignado, 404 por recurso ausente y 409 por transición o conflicto.

Método / ruta	Uso	Respuesta esperada
GET /api/appointments	Listar citas	200
POST /api/appointments	Crear cita	201 / 400 / 409
GET /api/appointments/{id}	Consultar una cita	200 / 404
POST /{id}/arrival	Registrar llegada	200 / 409
POST /{id}/decision	Decisión médica	200 / 403 / 409
POST /{id}/cancel	Cancelar cita	200 / 409
POST /{id}/no-show	Registrar inasistencia	200 / 409

8. Calidad y automatización

Hay pruebas unitarias de invariantes de dominio y pruebas de integración HTTP con perfil H2. JaCoCo está configurado con umbral de 80% de instrucciones en mvn verify; la cobertura real debe confirmarse ejecutando el build, no se afirma sin reporte. GitHub Actions ejecuta Maven verify en push y pull request. La ejecución del workflow queda pendiente de publicar el proyecto en un repositorio accesible.

9. Contenedores, configuración y despliegue

Docker Compose define PostgreSQL, aplicación y un perfil de ejecución de tests. .env.example separa configuración; no contiene secretos reales. Para despliegue se recomienda Railway, Render o plataforma equivalente con PostgreSQL administrado, variables DB_URL/DB_USER/DB_PASSWORD/PORT y Swagger activo. No se inventa una URL pública: requiere cuenta y credenciales del equipo.

10. Trazabilidad

La matriz docs/Traceability.md conecta reglas BR-01 a BR-09 con historias, clases, endpoints y pruebas. Señala explícitamente brechas: resultado clínico, historial e indicadores no están completos en esta versión del código; tampoco hay pruebas de integración de colisiones concurrentes. La matriz debe actualizarse tras completar esos módulos.

Regla	Historia	Componente	Estado
BR-01	HU-04	Appointment / Specification	Implementado
BR-03	HU-05	Service + Repository	Parcial; concurrencia pendiente
BR-05/06	HU-07/08	State Machine	Implementado
BR-08	HU-10	Domain Event	Evento en memoria
FR-11	HU-13	Resultado clínico	Pendiente
FR-12	HU-14	Historial	Pendiente
FR-13	HU-15	Indicadores	Pendiente

11. Riesgos y pendientes

El prototipo no debe exponerse con datos reales. Falta autenticación/autorización robusta; el identificador del médico se envía en la petición y no deriva de una identidad autenticada. La comprobación de solapamientos no basta para resolver carreras concurrentes sin restricciones/aislamiento adicional. También falta migración de esquema versionada, auditoría persistente inmutable, UI operativa, módulos completos de pacientes/médicos/sucursales y panel de indicadores.

12. Plan de demostración y defensa

Demostrar Swagger; crear cita con intervalo válido; confirmar; registrar llegada con retraso; observar minutos calculados y ausencia de decisión automática; registrar decisión como médico asignado; probar decisión de otro médico y transición inválida; cancelar otra cita y demostrar evento; ejecutar mvn verify; iniciar Compose. Explicar cómo las Specifications aplican reglas, cómo la máquina de estados evita cambios inválidos y cómo la matriz conecta requerimientos con pruebas.

indicadores, seguridad productiva, despliegue real y evidencia de cobertura deben completarse y verificarse antes de afirmar cumplimiento total del PIA.

14. Integración del análisis de negocio

Esta sección consolida los artefactos previos de MediNova y los vincula con la implementación contenida en MediNova_PIA. El caso describe una clínica privada con tres sucursales y aproximadamente 35 médicos, algunos empleados y otros por honorarios; por ello los horarios pueden cambiar semanalmente. Las solicitudes de cita llegan por teléfono, WhatsApp o recepción. El sistema propuesto prioriza una agenda consistente, la prevención de traslapes y el registro auditable de excepciones.

14.1 Contexto, situación actual y oportunidad

Dimensión	Situación documentada	Necesidad del sistema
Canales	Teléfono, WhatsApp y recepción presencial.	Concentrar el registro y la consulta de citas.
Agenda	Agenda compartida consultada por médico, especialidad y sucursal.	Consultar disponibilidad de forma consistente.
Capacidad	Tres sucursales y alrededor de 35 médicos; horarios variables.	Administrar disponibilidad y evitar conflictos.
Paciente	Alta de nuevos pacientes; búsqueda por teléfono o expediente.	Identificación y reutilización del registro.
Excepciones	Tardanzas, cancelaciones, no-show y reprogramaciones.	Estados explícitos y eventos de seguimiento.
Aseguradora	Convenios y autorización previa para ciertos servicios.	Validar autorización cuando sea requisito.
Gestión	Dificultad para medir no-shows e historial.	Trazabilidad e indicadores administrativos.

14.2 Actores y responsabilidades

Actor	Responsabilidad en el negocio	Interacción contemplada
Paciente	Solicita cita, proporciona datos y acude o puede no presentarse.	Origen de solicitud y atención.
Recepcionista	Consulta agenda, registra pacientes/citas, confirma, registra llegada y gestiona reprogramación.	Principal operador de la agenda.
Médico	Proporciona horarios, decide sobre tardanzas y realiza la consulta.	Autoridad de decisión sobre atención tardía.
Personal administrativo	Participa en gestión administrativa.	Consulta de indicadores e historial (brecha funcional).
Aseguradora	Interviene en convenios y autorización previa.	Validación de autorización cuando aplica.

14.3 Objetivos y alcance

Tipo	Definición
Objetivo general	Implementar una solución demostrativa para gestionar el ciclo de vida de las citas de MediNova con reglas explícitas, trazabilidad y API documentada.
Objetivo específico 1	Validar datos y condiciones de agendamiento mediante Specification.
Objetivo específico 2	Controlar transiciones mediante una máquina de estados encapsulada en Appointment.
Objetivo específico 3	Registrar llegada, tardanza y decisión del médico asignado sin automatizar la decisión clínica.
Objetivo específico 4	Persistir citas y exponer operaciones por API REST con documentación OpenAPI.
Incluido	Alta y consulta básica de pacientes/médicos, creación/listado/detalle de citas, confirmación, llegada, decisión, cancelación y no-show.
Fuera de alcance actual	Expediente clínico integral, recetas/estudios, historial clínico completo, indicadores, identidad productiva, portal de aseguradoras y despliegue público.

15. Requerimientos del negocio (BRD)

El BRD de MediNova identifica la gestión de citas, consultas y no-shows como proceso central. Se mantienen los hechos del caso y se evita convertir en regla definitiva cualquier política que el stakeholder no haya especificado.

15.1 Reglas de negocio refinadas

ID	Regla	Estado	Evidencia / observación
BR-01	Una cita debe relacionar paciente, médico, sucursal, especialidad, fecha/hora y motivo.	Parcial/implementado	Appointment + validaciones de creación.
BR-02	El médico debe corresponder a la especialidad seleccionada.	Implementado	AppointmentService.create compara especialidad con Doctor.
BR-03	No deben existir traslapes activos para el mismo médico.	Parcial	Consulta de traslape en AppointmentRepository; falta protección concurrente/índice de exclusión.
BR-04	Una cita solicitada puede confirmarse una sola vez.	Implementado	SOLICITADA → CONFIRMADA.
BR-05	La llegada registra hora real y minutos de tardanza.	Implementado	registerArrival calcula minutos sin valor negativo.
BR-06	La tardanza no decide automáticamente la atención; decide el médico asignado.	Implementado en núcleo	ATENDER, ESPERAR o REPROGRAMAR; autenticación real pendiente.
BR-07	Si se requiere autorización de aseguradora, debe existir autorización.	Parcial	Specification recibe bandera de requisito y autorización.
BR-08	La cancelación debe conservar motivo y evento.	Parcial	Estado y evento de dominio en memoria; persistencia de auditoría durable pendiente.
BR-09	El no-show solo procede desde estados solicitada/confirmada.	Implementado	State Machine valida transición.
BR-10	Las políticas de sobrecupo y reincidencia deben ser por médico/política institucional.	Pendiente de validación	No se inventan límites ni sanciones.

15.2 Requerimientos funcionales

ID	Requerimiento	Estado	Evidencia
RF-01	Registrar paciente nuevo con datos básicos.	Parcial	POST /api/patients; modelo disponible, campos requeridos del caso no todos están completos.
RF-02	Localizar paciente por teléfono o expediente.	Parcial	Catálogo REST básico; búsqueda avanzada no confirmada.
RF-03	Consultar disponibilidad por médico/especialidad/sucursal.	Pendiente	No hay endpoint dedicado de disponibilidad.
RF-04	Agendar cita con datos obligatorios.	Implementado	POST /api/appointments.
RF-05	Prevenir traslapes de agenda.	Parcial	Validación de solapamiento en servicio.
RF-06	Confirmar cita.	Implementado	POST /{id}/confirm.
RF-07	Registrar llegada y tardanza.	Implementado	POST /{id}/arrival.
RF-08	Registrar decisión médica.	Implementado	POST /{id}/decision.
RF-09	Cancelar cita con motivo.	Implementado	POST /{id}/cancel.
RF-10	Registrar no-show.	Implementado	POST /{id}/no-show.
RF-11	Registrar resultado clínico.	Pendiente	No existe módulo de recetas/estudios.
RF-12	Consultar historial del paciente.	Pendiente	Identificado en trazabilidad como pendiente.
RF-13	Consultar indicadores administrativos.	Pendiente	No existe endpoint de métricas.
RF-14	Gestionar horarios semanales y sobrecupo.	Pendiente	No existe módulo completo de agenda.

15.3 Requerimientos no funcionales

ID	Requisito	Evidencia/estado
RNF-01	API REST documentada con OpenAPI.	openapi.yaml y springdoc configurado.

RNF-02	Persistencia relacional.	Spring Data JPA y PostgreSQL 16 en Compose.
RNF-03	Pruebas automatizadas.	JUnit/Spring Boot Test; ejecución de verificación no acreditada en este entorno.
RNF-04	Cobertura mínima de 80%.	JaCoCo con umbral configurado; cobertura real pendiente de reporte.
RNF-05	Configuración externa.	.env.example y variables DB_URL, DB_USER, DB_PASSWORD, PORT.
RNF-06	Entorno reproducible.	Dockerfile y docker-compose.yml.
RNF-07	CI/CD.	Workflow GitHub Actions definido; ejecución remota activa no acreditada.
RNF-08	Protección de datos.	Uso exclusivo de datos ficticios; autenticación/autorización productiva pendiente.

16. Historias de usuario y criterios de aceptación Gherkin

El paquete de análisis contiene 20 historias de usuario, por encima del mínimo de 15 solicitado. Los escenarios siguientes conservan la intención de UserStories_Gherkin.md; cada historia se relaciona con un estado de implementación para evitar confundir requerimiento con funcionalidad disponible.

ID	Historia	Como / quiero	Criterio Gherkin: camino válido	Criterio Gherkin: excepción	Estado
HU-01	Registrar paciente	Recepcionista registra un paciente nuevo.	Dado nombre, teléfono, fecha de nacimiento y correo válidos, cuando guarda, entonces el paciente queda registrado.	Dado que falta un dato obligatorio, cuando guarda, entonces se rechaza.	Parcial
HU-02	Localizar paciente	Recepcionista busca por teléfono o expediente.	Dado un teléfono o expediente existente, cuando busca, entonces aparece el paciente coincidente.	Dado que no hay coincidencia, cuando busca, entonces se indica que puede registrarse un paciente nuevo.	Parcial
HU-03	Consultar disponibilidad	Recepcionista consulta por médico, especialidad y sucursal.	Dado rango y filtros válidos, cuando consulta, entonces solo aparecen espacios disponibles.	Dado un filtro incompleto, cuando consulta, entonces se solicita completar los datos.	Pendiente
HU-04	Agendar cita	Recepcionista crea una cita con datos completos.	Dada una solicitud válida y sin traslape, cuando guarda, entonces se crea en SOLICITADA.	Dado un intervalo inválido, cuando guarda, entonces se rechaza con explicación.	Implementado
HU-05	Prevenir duplicidad	Recepcionista evita solapamientos.	Dado que el médico tiene una cita activa en el intervalo, cuando intenta agendar otra, entonces se rechaza por conflicto.	Dado un intervalo libre, cuando agenda, entonces continúa validaciones.	Parcial
HU-06	Confirmar cita	Recepcionista registra confirmación.	Dada una cita solicitada, cuando confirma, entonces pasa a CONFIRMADA y queda evento.	Dada una cita en estado no confirmable, cuando confirma, entonces se rechaza.	Implementado
HU-07	Registrar llegada	Recepcionista registra hora real.	Dada una cita solicitada o confirmada, cuando registra llegada, entonces se guarda hora y minutos de retraso.	Dado que la llegada ya fue registrada o la cita está cancelada, cuando repite, entonces se rechaza.	Implementado
HU-08	Decisión de tardanza	Médico asignado decide atender, esperar o reprogramar.	Dada una llegada registrada, cuando el médico asignado decide ATENDER, ESPERAR o REPROGRAMAR, entonces se registra la decisión.	Dada una tardanza de 15 minutos o menos, el sistema no decide automáticamente y espera decisión médica.	Implementado
HU-09	Respetar autoridad médica	Solo el médico asignado decide la atención.	Dada una cita con llegada, cuando decide el médico asignado, entonces se acepta.	Dada la solicitud de otro médico, cuando decide, entonces se rechaza.	Parcial: validación por ID; falta identidad autenticada.
HU-10	Cancelar cita	Recepcionista cancela con motivo.	Dada una cita cancelable, cuando cancela con motivo, entonces queda CANCELADA y genera evento.	Dada una cita atendida o cancelada, cuando intenta cancelarla, entonces se rechaza.	Implementado
HU-11	Registrar no-show	Recepcionista marca inasistencia.	Dada una cita solicitada o confirmada, cuando marca no-show, entonces queda NO_SHOW.	Dada una cita atendida o cancelada, cuando marca no-show, entonces se rechaza.	Implementado
HU-12	Autorizar aseguradora	Recepcionista valida autorización previa.	Dado que el servicio requiere autorización y esta fue concedida, cuando valida, entonces la especificación se cumple.	Dado que se requiere autorización y no existe, cuando valida, entonces se rechaza.	Parcial
HU-13	Registrar resultado	Médico registra receta, estudios o seguimiento.	Dada una cita atendida, cuando registra resultado, entonces se asocia a cita y paciente.	Dada una cita no atendida, cuando intenta registrar resultado, entonces se solicita completar atención primero.	Pendiente
HU-14	Consultar historial	Personal autorizado consulta historial.	Dado un identificador de paciente, cuando consulta historial, entonces se muestran citas y estados ordenados por fecha.	Dado un identificador inexistente, cuando consulta, entonces se informa que no hay registro.	Pendiente
HU-15	Consultar indicadores	Administración consulta métricas.	Dado un periodo válido, cuando solicita indicadores, entonces entrega conteos por estado y sucursal.	Dado un periodo inválido, cuando solicita, entonces devuelve error de validación.	Pendiente

HU-16	Gestionar agenda	Médico actualiza horarios semanales.	Dado un horario válido, cuando se publica, entonces queda disponible para búsquedas futuras.	Dado un horario que se solapa con uno existente, cuando se publica, entonces se advierte conflicto.	Pendiente
HU-17	Reprogramar cita	Recepcionista crea nueva cita de seguimiento.	Dada una cita REQUIERE_REPROGRAMACION, cuando crea una nueva cita libre, entonces conserva relación de seguimiento.	Dado que el nuevo horario no está disponible, cuando reprograma, entonces no confirma la nueva cita.	Parcial
HU-18	Revisar sobrecupo	Recepción respeta política individual del médico.	Dado un médico que permite sobrecupo y existe capacidad definida, cuando se solicita, entonces valida límite.	Dado un médico que no permite sobrecupo, cuando se solicita uno, entonces se rechaza.	Pendiente
HU-19	Gestionar autorización	Recepción registra respuesta de aseguradora.	Dada una respuesta de autorización, cuando la registra, entonces queda asociada a cita/servicio.	Dado que el procedimiento no requiere autorización, cuando valida, entonces no exige respuesta.	Pendiente/parcial
HU-20	Auditar cambios	Administración conserva eventos de cambios.	Dado un cambio de dominio, cuando se aplica, entonces genera evento con tipo, cita y momento.	Dada una transición inválida, cuando se intenta, entonces no genera evento de éxito.	Parcial: eventos transitorios.

16.1 Priorización funcional para la demostración

Prioridad	Historias	Motivo
P0 — núcleo demostrable	HU-04, HU-05, HU-06, HU-07, HU-08, HU-09, HU-10, HU-11	Constituyen el ciclo principal de agenda, llegada y decisión.
P1 — catálogos y reglas	HU-01, HU-02, HU-12, HU-19, HU-20	Soportan datos maestros, autorización y trazabilidad.
P2 — ampliaciones	HU-03, HU-13, HU-14, HU-15, HU-16, HU-17, HU-18	Requieren módulos adicionales o políticas aprobadas.

17. Proceso de negocio y BPMN

El proceso AS-IS documentado inicia con una solicitud por teléfono, WhatsApp o recepción; recepción busca disponibilidad en agenda compartida por médico, especialidad y sucursal; identifica al paciente por teléfono o expediente o registra uno nuevo; captura los datos de la cita y realiza un intento de confirmación un día antes. En la llegada se evalúa el retraso: hasta 15 minutos se consulta al médico y por encima de 15 minutos normalmente se ofrece reagendar. La decisión final corresponde al médico. Después de la consulta puede emitirse receta, solicitarse estudios o indicarse una nueva cita.

Etapa	Actor principal	Entrada	Salida / decisión
Solicitud	Paciente	Necesidad de consulta	Solicitud por canal disponible.
Búsqueda	Recepcionista	Médico, especialidad, sucursal y rango	Espacios disponibles identificados.
Identificación	Recepcionista	Teléfono o expediente	Paciente existente o alta de nuevo.
Agendamiento	Recepcionista	Paciente, médico, sucursal, hora, motivo y aseguradora	Cita solicitada.
Confirmación	Recepcionista	Cita próxima	Intento de confirmación un día antes.
Llegada	Recepcionista	Hora de llegada real	Minutos de tardanza calculados.
Decisión tardanza	Médico asignado	Hora de llegada y agenda	ATENDER / ESPERAR / REPROGRAMAR.
Consulta	Médico	Paciente atendido	Receta, estudios o nueva cita.
Cierre/excepción	Recepción/administración	Cancelación o ausencia	Estado cancelado/no-show y seguimiento.

17.1 Regla de tardanza

El caso base establece que hasta 15 minutos de retraso recepción consulta al médico; cuando se superan los 15 minutos normalmente se ofrece reagendar. La implementación calcula minutos de tardanza y registra llegada, pero no codifica una decisión automática basada en el umbral: el médico asignado decide explícitamente. El valor de 15 minutos se trata como regla del proceso documentado y debe confirmarse si la institución desea excepciones por especialidad o médico.

17.2 Diagramas de proceso y secuencia

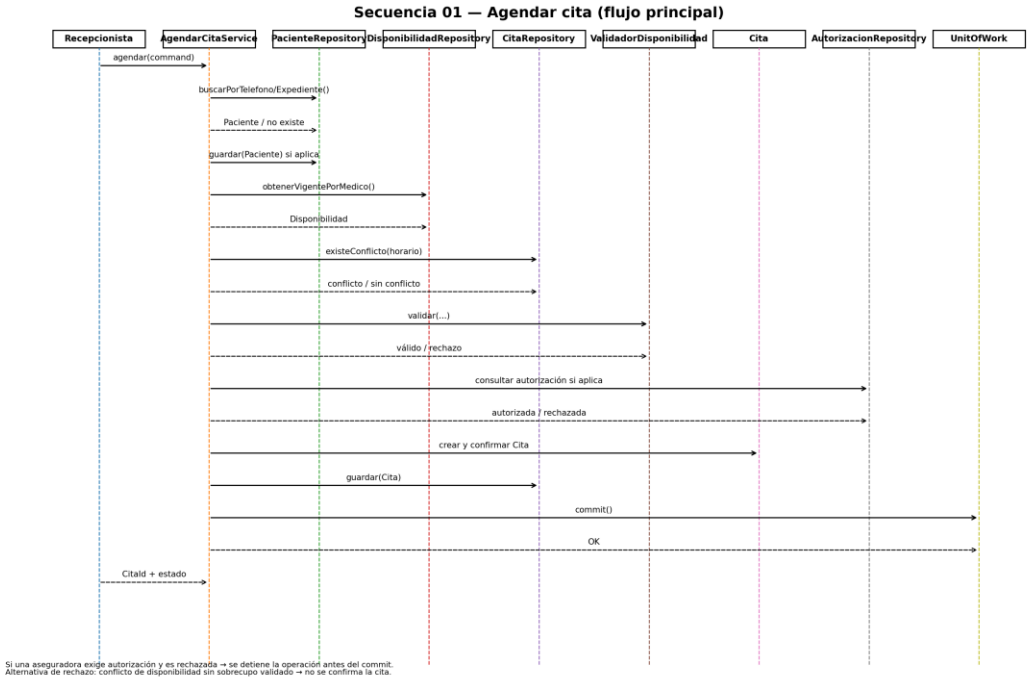


Figura 1. Secuencia de agendamiento de cita.

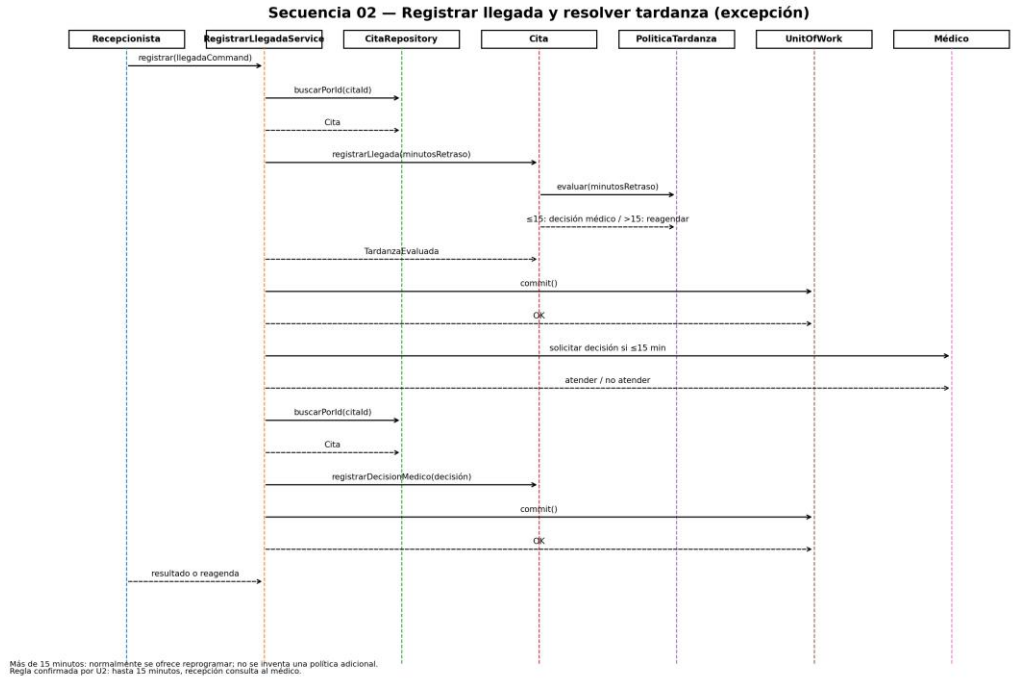


Figura 2. Secuencia de llegada y decisión ante tardanza.

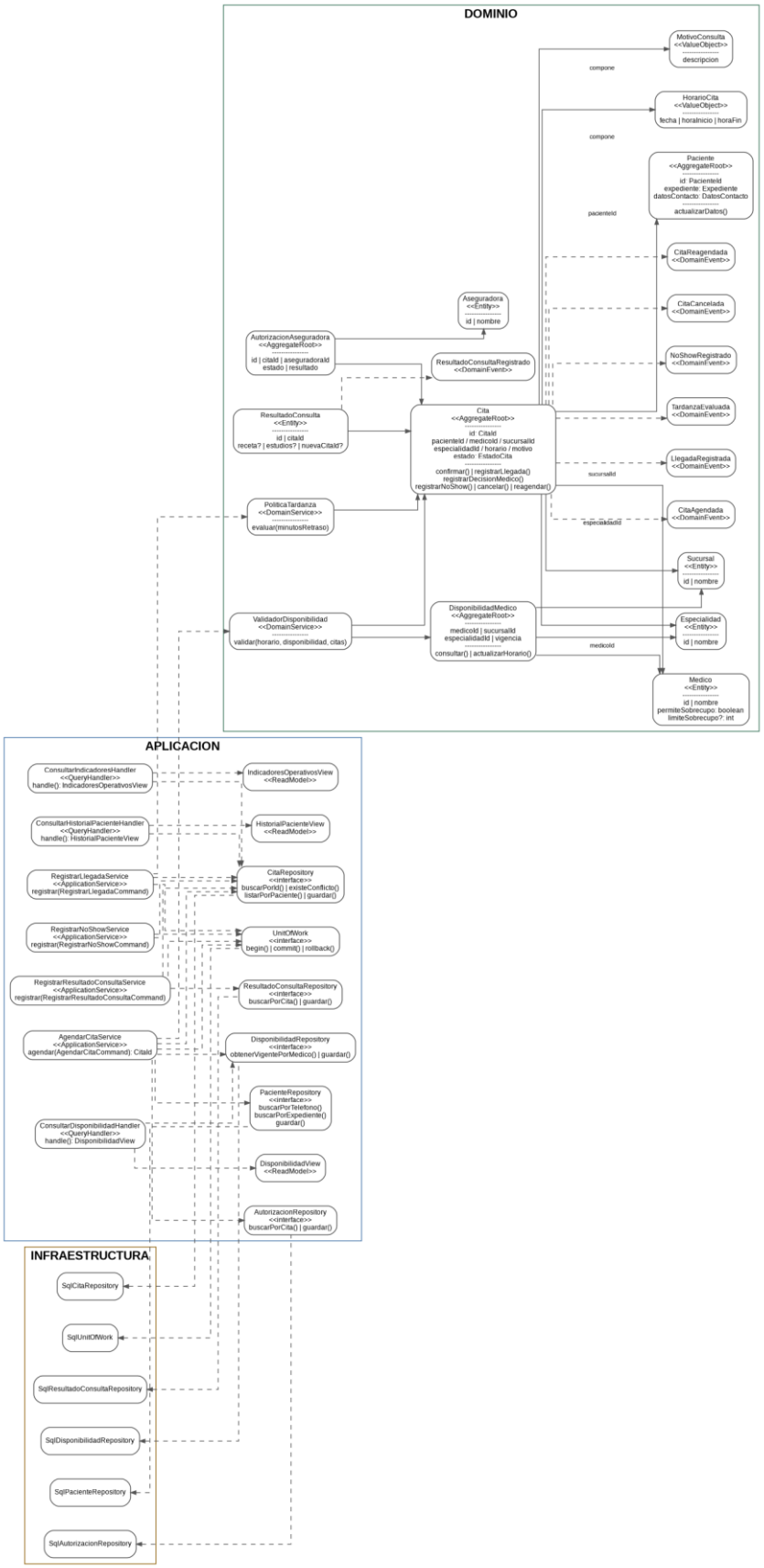


Figura 3. Diagrama de clases DDD.

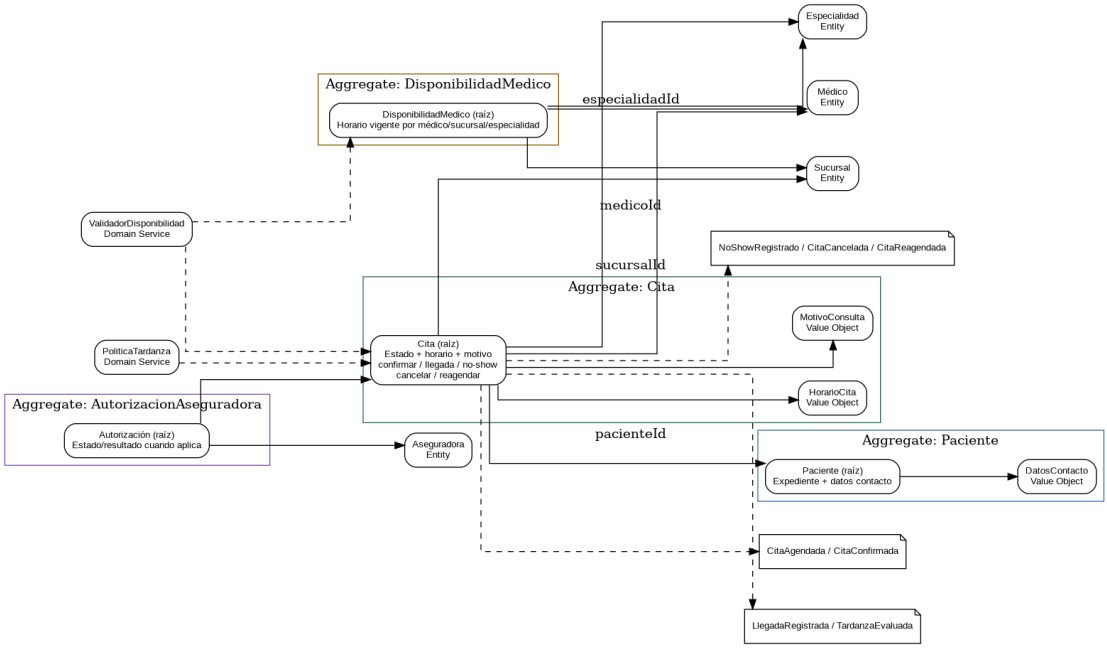


Figura 4. Mapa de dominio.

18. Arquitectura de software y diseño DDD

La solución aplica un modelo de dominio centrado en Appointment (Cita) como agregado raíz del ciclo de vida de la cita. La entidad protege invariantes de fechas, estados y autoridad del médico; AppointmentService coordina casos de uso; los repositorios abstraen persistencia; los controladores traducen HTTP a operaciones de aplicación.

Capa	Paquete / componente	Responsabilidad	Clases / evidencia
API	mx.medinova.api	Exponer recursos REST, validar payload y devolver códigos HTTP.	AppointmentController, PatientController, DoctorController, ApiExceptionHandler.
Aplicación	mx.medinova.application	Orquestar transacciones y casos de uso.	AppointmentService.
Dominio	mx.medinova.domain	Entidades, estados, especificaciones, decisiones y eventos.	Appointment, AppointmentStatus, MedicalDecision, Specification, AppointmentSpecifications, DomainEvent.
Infraestructura	mx.medinova.infrastructure	Persistencia JPA y carga inicial.	AppointmentRepository, PatientRepository, DoctorRepository, DataInitializer.

18.1 Modelo de dominio

Elemento DDD	Tipo	Responsabilidad / regla
Appointment	Entidad y Aggregate Root	Controla estado, intervalo, llegada, tardanza, decisión y cancelación.
AppointmentStatus	Value type enumerado	SOLICITADA, CONFIRMADA, LLEGADA_REGISTRADA, ATENDIDA, REQUIERE_REPROGRAMACION, CANCELADA, NO_SHOW.
MedicalDecision	Enumeración de dominio	ATENDER, ESPERAR, REPROGRAMAR.
Specification<T>	Abstracción de regla	Composición AND, OR y NOT.
AppointmentSpecifications	Especificaciones de negocio	Paciente/médico/sucursal activos, autorización y validez del intervalo.
DomainEvent	Evento de dominio	Representa confirmación, llegada, decisión, cancelación y no-show.
AppointmentService	Servicio de aplicación	Coordina persistencia y validaciones entre agregado y repositorios.
AppointmentRepository	Repositorio	Consulta y guarda citas, incluyendo búsqueda de traslape.

18.2 Límites y consistencia del agregado

El agregado Appointment contiene los datos de referencia patientId, doctorId y branchId, especialidad, intervalo, motivo, estado y metadatos de llegada/decisión. La entidad no carga objetos completos de paciente o médico como asociaciones JPA; el servicio valida existencia del paciente y médico y correspondencia de especialidad antes de guardar. Esta decisión mantiene el agregado enfocado en el ciclo de cita, pero requiere fortalecer integridad referencial y transacciones para producción.

18.3 Máquina de estados

Estado origen	Acción	Estado destino	Condición
SOLICITADA	confirm()	CONFIRMADA	Solo se confirma una cita solicitada.
SOLICITADA / CONFIRMADA	registerArrival()	LLEGADA_REGISTRADA	Hora de llegada obligatoria.
LLEGADA_REGISTRADA	decide(ATENDER)	ATENDIDA	Solo médico asignado.

LLEGADA_REGISTRADA	decide(ESPERAR)	LLEGADA_REGISTRADA	Se conserva estado de espera.
LLEGADA_REGISTRADA	decide(REPROGRAMAR)	REQUIERE_REPROGRAMACION	Solo médico asignado.
SOLICITADA / CONFIRMADA / LLEGADA_REGISTRADA / REQUIERE_REPROGRAMACION	cancel()	CANCELADA	No permite cancelar atendida, cancelada o no-show.
SOLICITADA / CONFIRMADA	markNoShow()	NO_SHOW	No-show no permitido desde otros estados.

19. Patrones de diseño y principios SOLID

Patrón/principio	Aplicación en MediNova	Beneficio	Límite / mejora
Domain Model / DDD	Appointment concentra comportamiento e invariantes.	Evita modelo anémico y reglas dispersas.	Ampliar agregados y Value Objects donde el dominio lo requiera.
Repository	AppointmentRepository y repositorios de catálogos.	Aísla persistencia del servicio de aplicación.	Definir contratos/interfaces de dominio para desacoplamiento mayor.
Service Layer	AppointmentService coordina creación, confirmación, llegada y decisión.	Centraliza transacciones y orquestación.	Mantener controladores delgados.
Specification	Reglas composables AND/OR/NOT.	Reutilización y pruebas aisladas.	Alinear todos los parámetros con estado persistido, no solo flags recibidos.
State Machine	Métodos de Appointment restringen transiciones.	Evita estados arbitrarios.	Agregar pruebas de cada transición y tabla de transiciones.
Domain Events	Eventos creados por cambios de estado.	Facilita auditoría e integración futura.	Actualmente lista transitoria @Transient; falta outbox o persistencia.
SOLID — SRP	Entidad gestiona invariantes; servicio orquesta; controlador expone HTTP.	Responsabilidades diferenciadas.	Reducir clases compactadas y métodos con múltiples responsabilidades.
SOLID — OCP	Specification permite agregar reglas componibles.	Extensión sin reescribir toda la validación.	Introducir catálogo de políticas y pruebas.
SOLID — LSP/ISP/DIP	Specification como contrato funcional; repositorios Spring inyectados.	Dependencias explícitas y contratos pequeños.	DIP se fortalecería con interfaces propias y adaptadores.

19.1 Justificación de selección

El problema de MediNova contiene reglas que dependen de contexto (paciente, médico, sucursal, intervalo y autorización) y cambios de estado sujetos a autoridad. Specification permite componer predicados de agendamiento; State Machine evita transiciones inválidas; Domain Model mantiene las reglas cercanas a la entidad que protege. Repository y Service Layer separan acceso a datos y coordinación de casos de uso.

20. Registros de decisiones de arquitectura (ADR)

ADR-001 — Java 21 y Spring Boot

Campo	Registro
Contexto	se requiere API mantenible y desplegable con persistencia relacional.
Decisión	Java 21, Spring Boot 3.4.5, Spring Web, Validation y Spring Data JPA.
Alternativas	Node.js/Express y Python/FastAPI.
Consecuencias	ecosistema maduro, estructura por capas y soporte de pruebas; requiere JDK/Maven.

ADR-002 — PostgreSQL como persistencia

Campo	Registro
Contexto	citas, pacientes y médicos requieren almacenamiento relacional.
Decisión	PostgreSQL 16 en Docker Compose; H2 para pruebas.
Alternativas	MySQL y SQLite.
Consecuencias	integridad y transacciones robustas; requiere credenciales, volumen persistente y migraciones.

ADR-003 — Appointment como agregado raíz

Campo	Registro
Contexto	el ciclo de cita concentra reglas y transiciones.
Decisión	Appointment controla invariantes y estados; referencias a paciente/médico/sucursal se manejan por ID.
Alternativas	lógica en controllers o CRUD sin invariantes.
Consecuencias	reglas testeables; integridad referencial y consistencia concurrente deben reforzarse.

ADR-004 — Specification composable

Campo	Registro
Contexto	las validaciones combinan estado activo, autorización e intervalo.
Decisión	Specification<T> con AND/OR/NOT y reglas nombradas.
Alternativas	condicionales duplicados en endpoints.
Consecuencias	composición y pruebas aisladas; alinear valores de entrada con entidades persistidas.

ADR-005 — State Machine en Appointment

Campo	Registro
Contexto	estados no deben cambiar arbitrariamente.
Decisión	transiciones encapsuladas en métodos de la entidad.
Alternativas	actualización libre de status.
Consecuencias	rechaza transiciones inválidas y facilita pruebas; se requiere completar cobertura de estados.

ADR-006 — API REST y OpenAPI

Campo	Registro
Contexto	los casos de uso deben demostrarse y tener contrato navegable.
Decisión	endpoints REST bajo /api y springdoc-openapi/Swagger UI.
Alternativas	interfaz web solamente.
Consecuencias	integración y prueba sencilla; endpoints requieren autenticación antes de datos reales.

ADR-007 — Docker Compose y CI

Campo	Registro
Contexto	entorno reproducible y verificación continua.
Decisión	Compose para DB/App/tests y GitHub Actions con mvn verify.
Alternativas	ejecución manual local.
Consecuencias	automatización reproducible; pipeline activo depende de repositorio remoto y ejecución comprobada.

Estado general de los ADR: aceptados para prototipo académico. La aceptación no equivale a certificación de producción; las consecuencias y pendientes se conservan como parte de la decisión.

21. API REST y persistencia

Método y ruta	Propósito	Respuesta esperada / observación
GET /api/appointments	Listar citas.	200; consulta de citas.
GET /api/appointments/{id}	Consultar detalle.	200 / 404.
POST /api/appointments	Crear cita.	201 / 400 / 409; valida IDs, especialidad, Specification y traslape.
POST /api/appointments/{id}/confirm	Confirmar cita.	200 / 409.
POST /api/appointments/{id}/arrival	Registrar hora real de llegada.	200 / 409; calcula tardanza.
POST /api/appointments/{id}/decision	Decisión del médico asignado.	200 / 403 / 409.
POST /api/appointments/{id}/cancel	Cancelar con motivo.	200 / 409.
POST /api/appointments/{id}/no-show	Registrar inasistencia.	200 / 409.
GET/POST/DELETE /api/patients	Operaciones básicas de pacientes.	Disponibilidad confirmada por pruebas de integración.
GET/POST/DELETE /api/doctors	Operaciones básicas de médicos.	Disponibilidad confirmada por pruebas de integración.

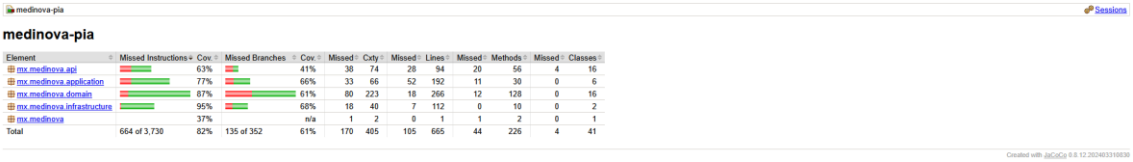
OpenAPI 3.0.3 se encuentra en docs/api/openapi.yaml. La interfaz Swagger UI está configurada mediante springdoc y se espera en /swagger-ui.html al iniciar la aplicación. La especificación OpenAPI incluida describe principalmente endpoints de citas; debe mantenerse sincronizada al ampliar catálogos, seguridad y nuevos casos de uso.

21.1 Persistencia

La aplicación utiliza Spring Data JPA con PostgreSQL como motor previsto en ejecución y H2 para el perfil de pruebas. Appointment persiste el identificador de paciente, médico y sucursal, especialidad, intervalo, motivo, estado, llegada, tardanza y decisión. Los eventos de dominio están marcados como transitorios y no se conservan como bitácora durable en esta versión.

22. Calidad, pruebas y CI/CD

Prueba / componente	Evidencia en repositorio	Qué valida	Resultado acreditado
AppointmentTest	src/test/java/.../domain/AppointmentTest.java	Intervalo inválido, tardanza, médico asignado, decisión previa a llegada, cancelación, Specification y confirmación.	Casos implementados;
AppointmentApiIntegrationTest	src/test/java/.../AppointmentApiIntegrationTest.java	HTTP list/detail, recursos estáticos, CRUD básico de pacientes/médicos, validación de IDs/especialidad y creación de cita.	Casos implementados;
JaCoCo	pom.xml	Umbral de 80% de instrucciones en verify.	Configuración presente.
GitHub Actions	/.github/workflows/ci.yml	Checkout, Java 21 y mvn -B verify en push/PR.	Workflow definido;
Docker Compose	docker-compose.yml	PostgreSQL, App y perfil tests.	Archivo presente;



22.1 Criterio de cobertura

El POM configura JaCoCo 0.8.12 para preparar agente, generar reporte en verify y aplicar un mínimo de 0.80 de cobertura de instrucciones a nivel de bundle. El umbral configurado no prueba por sí mismo que la cobertura alcanzada sea ≥80 %. Para acreditarlo se requiere ejecutar mvn -B verify y anexar target/site/jacoco/index.html o el XML de JaCoCo.

22.2 Pipeline

El workflow .github/workflows/ci.yml se activa en push a main/master y en pull_request. Configura Temurin Java 21 con caché Maven y ejecuta mvn -B verify. La ejecución verde solo puede afirmarse después de publicar el repositorio en GitHub y observar el resultado del workflow.

23. Contenerización, configuración y despliegue

Elemento	Configuración observada	Uso
Dockerfile	Construcción de la aplicación Java.	Imagen de la app.
docker-compose.yml	Servicios db (postgres:16-alpine), app y tests bajo profile test.	Entorno reproducible.
.env.example	Plantilla de variables de conexión y puerto.	Separar configuración del código.
Variables	DB_NAME, DB_USER, DB_PASSWORD, DB_URL, APP_PORT/PORT.	Configuración por entorno.
Volumen	pgdata para /var/lib/postgresql/data.	Persistencia local de PostgreSQL.
Despliegue	Railway/Render/Fly.io o equivalente sugerido.	URL pública no proporcionada; despliegue pendiente.

No se debe subir el archivo .env con secretos reales. El archivo .env.example es la referencia segura para configurar el entorno. Para una demostración remota se requiere aprovisionar PostgreSQL, configurar variables de entorno, publicar la imagen/aplicación, verificar health/readiness y probar Swagger UI en producción.

24. Trazabilidad y matriz de cumplimiento

Regla/historia	Dominio/caso de uso	API	Prueba/evidencia	Estado
BR-01 / HU-04	Appointment constructor + Specification	POST /api/appointments	AppointmentTest.rejectsInvalidInterval	Implementado
BR-02 / HU-04	AppointmentService valida especialidad	POST /api/appointments	Integration: specialty mismatch	Implementado
BR-03 / HU-05	AppointmentService + overlap query	POST /api/appointments	Prueba concurrente pendiente	Parcial
BR-04 / HU-06	Appointment.confirm / State Machine	POST /{id}/confirm	confirmationRequiresRequested	Implementado
BR-05 / HU-07	Appointment.registerArrival	POST /{id}/arrival	latenessDoesNotAutoDecide	Implementado
BR-06 / HU-08/HU-09	Appointment.decide	POST /{id}/decision	assignedDoctorCanDecide / otherDoctorCannotDecide	Parcial: autenticación pendiente
BR-07 / HU-12/HU-19	authorizationSatisfied Specification	POST /api/appointments	specificationsCombineAndNot	Parcial
BR-08 / HU-10/HU-20	Appointment.cancel + DomainEvent	POST /{id}/cancel	cancellationEmitsEvent	Parcial: evento transitorio
BR-09 / HU-11	Appointment.markNoShow	POST /{id}/no-show	Prueba unitaria pendiente	Implementado en dominio
RF-11 / HU-13	Módulo de resultados clínicos	No disponible	No disponible	Pendiente
RF-12 / HU-14	Historial del paciente	No disponible	No disponible	Pendiente
RF-13 / HU-15	Indicadores administrativos	No disponible	No disponible	Pendiente
RF-14 / HU-16/HU-18	Agenda y sobrecupo	No disponible	No disponible	Pendiente

24.1 Lectura de cumplimiento

El núcleo de citas está conectado con entidades, servicios, endpoints y pruebas. La trazabilidad identifica explícitamente brechas en lugar de asignar cumplimiento ficticio. Las historias HU-13 a HU-16 y HU-18 requieren ampliaciones de código y pruebas; HU-17 requiere relación de seguimiento persistente; HU-20 requiere almacenamiento durable de eventos.

25. Riesgos, seguridad y limitaciones

Riesgo	Impacto	Mitigación recomendada	Estado
Sin autenticación y roles productivos	Acceso indebido a datos y decisiones.	Spring Security/OIDC, roles PACIENTE/RECEPCION/MEDICO/ADMIN y autorización por recurso.	Pendiente
ID de médico enviado en request	No acredita identidad real del actor.	Obtener identidad desde token/sesión autenticada.	Pendiente
Eventos transitorios	Pérdida de auditoría al terminar la operación/proceso.	Persistencia de eventos/outbox transaccional.	Pendiente
Concurrencia de agendamiento	Dos solicitudes simultáneas podrían reservar mismo intervalo.	Transacción, bloqueo/constraint temporal y prueba concurrente.	Pendiente
Datos clínicos sensibles	Riesgo legal y de privacidad.	Solo datos ficticios en demo; cifrado, control de acceso y políticas de retención en producción.	Obligatorio antes de producción
Módulos faltantes	No cubre todo el ciclo clínico/administrativo.	Implementar historial, resultados, agenda e indicadores.	Pendiente
CI/deploy sin evidencia remota	No se acredita disponibilidad pública ni pipeline verde.	Publicar repositorio y despliegue, anexar capturas y URLs verificadas.	Pendiente

26. Guion de demostración en vivo

Paso	Acción de demostración	Evidencia que debe observarse
1	Iniciar `docker compose up --build -d` o ejecución local con PostgreSQL.	Contenedores/servicio arriba.
2	Abrir `http://localhost:8080/swagger-ui.html`.	Contrato OpenAPI navegable.
3	Consultar GET /api/appointments.	Respuesta 200.
4	Crear cita con paciente, médico, sucursal, especialidad, startAt/endAt y motivo válidos.	201 y estado SOLICITADA.
5	Intentar crear cita traslapada para el mismo médico.	Conflicto rechazado.
6	Confirmar cita.	Estado CONFIRMADA.
7	Registrar llegada posterior al inicio programado.	LLEGADA_REGISTRADA y minutos calculados.
8	Mostrar que el sistema no decide automáticamente la atención.	medicalDecision permanece nula hasta decisión.
9	Enviar decisión con médico asignado y después con otro ID.	Decisión válida y rechazo de actor no asignado.
10	Probar cancelación/no-show y transición inválida.	Estados y errores esperados.

27. Preguntas técnicas para la defensa

Pregunta	Respuesta sugerida
¿Por qué Appointment es Aggregate Root?	Porque concentra el ciclo de vida y las invariantes de una cita; los cambios de estado deben pasar por sus métodos.
¿Qué aporta Specification?	Permite combinar reglas independientes mediante AND/OR/NOT y probarlas sin depender del controlador.
¿Cómo se controla la máquina de estados?	Los métodos confirm, registerArrival, decide, cancel y markNoShow verifican el estado de origen y lanzan excepción si la transición no es válida.
¿Cómo se calcula la tardanza?	Se calcula la diferencia en minutos entre startAt y arrivedAt, limitada a cero si llega antes. No decide automáticamente la atención.
¿Quién decide una tardanza?	El médico asignado; la entidad compara el doctorId recibido con doctorId de la cita. La autenticación productiva sigue pendiente.
¿Cómo se previenen duplicidades?	Se consulta si existe una cita activa del médico cuyo intervalo se traslape; para producción se requiere protección concurrente a nivel DB.
¿Qué pruebas existen?	JUnit para reglas del dominio y Spring Boot Test/MockMvc para endpoints y validaciones. La ejecución y cobertura real deben anexarse.
¿El 80% está demostrado?	No con los archivos revisados: JaCoCo tiene umbral configurado, pero falta adjuntar un reporte de ejecución que demuestre el valor.
¿Está desplegado?	No hay URL pública acreditada en el paquete. Docker Compose define un entorno reproducible y la publicación remota está pendiente.
¿Qué falta para producción?	Autenticación/autorización, auditoría persistente, concurrencia, migraciones, privacidad, monitoreo, respaldo y completar módulos funcionales.

28. Conclusiones

El proyecto MediNova transforma el análisis de negocio de las Unidades 2 y 3 en una base de software demostrable. La solución implementa un núcleo de gestión de citas con Java 21, Spring Boot, PostgreSQL, DDD, Specification, State Machine, eventos de dominio, API REST y pruebas automatizadas definidas. Las historias de usuario, reglas y trazabilidad mantienen la relación entre el problema de negocio y los componentes.

La principal decisión de dominio consiste en mantener la autoridad sobre la atención tardía en el médico asignado, evitando que el sistema tome una decisión clínica automática. La máquina de estados protege el ciclo de vida de la cita y Specification concentra validaciones de agendamiento.

La entrega debe considerarse un prototipo académico funcional en su núcleo, no un sistema clínico productivo terminado. La cobertura real ≥80 %, ejecución del pipeline, repositorio remoto, despliegue público, seguridad de identidad, auditoría durable y módulos de historial, resultados e indicadores requieren evidencia o implementación adicional antes de declarar cumplimiento total.

29. Referencias y artefactos fuente

Artefacto	Uso en este reporte
PIA_CLN_Proyecto_Integrador_AD2026-1.pdf	Requisitos obligatorios y criterios de evaluación del PIA.
Eq01_MediNova_BRD1.pdf	Contexto, problema, objetivos y reglas de negocio.
Eq01_MediNova_UserStories.pdf	Historias de usuario, levantamiento y criterios de aceptación.
Eq01_MediNova_BPMN.pdf	Proceso de negocio AS-IS.
Eq01_MediNova_Glosario-1.pdf	Lenguaje ubicuo y conceptos del dominio.
Informe_Arquitectura_MediNova_Unidad3.pdf	Decisiones DDD, UML, patrones y trazabilidad previas.
MediNova_PIA/docs/BRD.md y UserStories_Gherkin.md	Requerimientos refinados del PIA.
MediNova_PIA/docs/architecture/ADRs.md	Decisiones de arquitectura implementadas.
MediNova_PIA/docs/api/openapi.yaml	Contrato de API.
MediNova_PIA/src y src/test	Implementación y pruebas revisadas.

30. Anexo: checklist final de evidencia

Entregable/rúbrica	Estado al revisar los archivos	Acción necesaria para cierre
BRD completo	Documento base disponible y refinado en este reporte.	Validar supuestos pendientes con stakeholder.
≥15 User Stories Gherkin	20 historias documentadas.	Completar automatización de historias pendientes.
BPMN y glosario	Artefactos fuente incluidos.	Adjuntar BPMN legible/exportado y validar políticas.
DDD/UML y patrones	Diagramas y código DDD presentes.	Mantener sincronía con versión final del código.
ADRs	7 ADRs en docs/architecture/ADRs.md.	Confirmar responsables/fechas/estado final.
Repositorio GitHub organizado	Estructura local cumple directorios mínimos.	Publicar URL y verificar historial de commits.
API/OpenAPI	OpenAPI y springdoc configurados.	Probar endpoints y Swagger en entorno final.
Pruebas/cobertura ≥80%	Pruebas escritas y JaCoCo configurado.	Ejecutar `mvn verify`
CI/CD activo	Workflow YAML presente.	Verificar ejecución verde en GitHub Actions.
Docker Compose DB + App + Tests	Compose define los servicios.	Ejecutar y guardar evidencia.
Variables de entorno	.env.example disponible.	Verificar que .env y secretos no se publiquen.
Despliegue y Swagger en producción	No se encontró URL pública.	Desplegar y anexar URL/capturas.
Defensa y demo	Guion y preguntas incluidos.	Realizar demo en vivo y registrar evidencia.